OPERATING SYSTEMS EXERCISE 2

Assignment – Closest Pair of Points

Implement a program which searches for the closest pair of points[1] in a set of 2D-points.

```
SYNOPSIS
    cpair
```

Instructions

The program takes an array of 2D points as input. The input is read from `stdin`. Each line consists of two floating point numbers, separated by whitespace, which represent the x and y coordinates of one point. The array ends when an EOF (End Of File) is encountered.

Your program must accept any number of points! Terminate the program with exit status `EXIT_FAILURE` if an invalid input is encountered.

The program computes the closest pair of points from its input values recursively, i.e. by calling itself:

1. If the array consists of only 1 point, then the program exits without generating any output.
2. If the array consists of two points, then write these to `stdout` and exit.
3. Otherwise the array consists of $n > 2$ points. Calculate x_m, the arithmetic mean of the x-coordinates of all the points. Divide the array into two parts, with one part containing all the points with an x-coordinate less than (or equal to) x_m and the other part containing all the points with an x-coordinate greater than x_m. [2]
4. Using *fork(2)* and *execlp(3)*, recursively execute this program in two child processes, one for each of the two parts. Use two unnamed pipes per child to redirect *stdin* and *stdout* (see *pipe(2)* and *dup2(3)*). Write one part to *stdin* of one child and the other part to *stdin* of the other child. Read the respective results from each child's *stdout*. The two child processes must run simultaneously!
5. Use *wait(2)* or *waitpid(2)* to read the exit status of the children. Terminate the program with exit status `EXIT_FAILURE` if the exit status of any of the two child processes is not `EXIT_SUCCESS`.
6. Let P_1 be the closest pair of the first part and P_2 the closest pair of the second part.
7. For each point in the first part, calculate its distance to each point in the second part. Remember the pair with the shortest distance as P_3.
8. Compare the distances of P_1, P_2 and P_3 and write the pair with the shortest distance to `stdout`, with the two points separated by a newline. Terminate the program with exit status `EXIT_SUCCESS`.

Throughout the program, it is sufficient to use single precision floating point numbers.

[1] `https://en.wikipedia.org/wiki/Closest_pair_of_points_problem`

[2] If all x-coordinates have the same value then this does not divide the array into two parts of equal length. It is not required to handle this special case. Your program's behavior may be undefined for such inputs (including infinite recursion).

Hints

1. You may use the header *math.h* for the distance calculations. Add `-lm` to the linker options to use these functions.
2. Create a structure to store the coordinates of a point.
3. In order to avoid endless recursion[3], fork only if the input number is greater than 1.
4. To output error messages and debug messages, always use *stderr* because *stdout* is redirected in most cases.

Examples

```
$ cat 1.txt
4.0 4.0
-1.0 1.0
1.0 -1.0
-4.0 -4.0
$ ./cpair < 1.txt
-1.000000 1.000000
1.000000 -1.000000
```

Small deviations of the resulting values, which are a consequence of the limited precision of floating point numbers, are not relevant and not considered to be an error.

[3] http://en.wikipedia.org/wiki/Fork_bomb

Bonus exercise, 5 points

Print the parent and child relations in form of a tree to stdout. Print all children which are forked from the parent. The tree should be readable at least to a depth of three. For every node, the intermediate result should be printed.
Depending on how often you call fork the wider the tree becomes. A simple tree example which searches for the maximum number in a set is shown below.

```
10
                         MAX(10,8,3,5,2,1,6,7)
                           /              \
                      MAX(10,8,3,5)    MAX(2,1,6,7)
                        /        \        /      \
                   MAX(10,8)  MAX(3,5)  MAX(2,1) MAX(6,7)
                    /    \      /   \     /   \    /   \
                   10    8     3     5   2     1   6    7
```

Instructions on how to print the tree

- leaf node:
  A leaf node should print the substep executed by it to stdout with a terminating newline.
- inner node:
  - To get the necessary identation use several blank characters. Think about a good way to find the right number of blank characters. For example you could use precalculated values or calculate the number from the first line you read from the children.
  - Calculate the intermediate result and print this and the executed operation to stdout.
  - Slash and backslash, which represent the branches of the tree, are printed to stdout.
  - Read the output from the children line by line via a pipe. This means read the first line from the first child, then the first line from the second child and so on. Remove the newline characters. Line up the results and then print it with a terminating newline to stdout. Do this for each line returned by the child.

Coding Rules and Guidelines

Your score depends upon the compliance of your submission to the presented guidelines and rules. Violations result in deductions of points. Hence, before submitting your solution, go through the following list and check if your program complies.

Rules

Compliance with these rules is essential to get any points for your submission. A violation of any of the following rules results in 0 points for your submission.

1. All source files of your program(s) must compile via

   ```
   $ gcc -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -D_SVID_SOURCE
         -D_POSIX_C_SOURCE=200809L -g -c filename.c
   ```

   without *errors* and your program(s) must link without *errors*. The compilation flags must be used in the Makefile. The feature test macros must not be bypassed (i.e., by undefining these macros or adding some in the C source code).

2. The functionality of the program(s) must conform to the assignment. The program(s) shall operate according to the specification/assignment given the test cases in the respective assignment.

General Guidelines

Violation of following guidelines leads to a deduction of points.

1. All source files of your program(s) must compile with

   ```
   $ gcc -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -D_SVID_SOURCE
         -D_POSIX_C_SOURCE=200809L -g -c filename.c
   ```

   without *warnings and info messages* and your program(s) must link without warnings.

2. There must be a Makefile implementing the targets: `all` to build the program(s) (i.e. generate executables) from the sources (this must be the first target in the Makefile); `clean` to delete all files that can be built from your sources with the Makefile.

3. The program shall operate according to the specification/assignment without major issues (e.g., segmentation fault, memory corruption).

4. Arguments have to be parsed according to UNIX conventions (we strongly encourage the use of `getopt(3)`). The program has to conform to the given synopsis/usage in the assignment. If the synopsis is violated (e.g., unspecified options or too many arguments), the program has to terminate with the usage message containing the program name and the correct calling syntax. Argument handling should also be implemented for programs without arguments.

5. Correct (=normal) termination, including a cleanup of resources.

6. Upon success the program has to terminate with exit code 0, in case of errors with an exit code greater than 0. We recommend to use the macros `EXIT_SUCCESS` and `EXIT_FAILURE` (defined in `stdlib.h`) to enable portability of the program.

7. If a function indicates an error with its return value, it *should* be checked in general. If the subsequent code depends on the successful execution of a function (e.g. resource allocation), then the return value *must* be checked.

8. Functions that do not take any parameters have to be declared with `void` in the signature, e.g., int get_random_int(void);.
9. Procedures (i.e., functions that do not return a value) have to be declared as `void`.
10. Error messages shall be written to `stderr` and should contain the program name `argv[0]`.
11. It is forbidden to use the functions: `gets`, `scanf`, `fscanf`, `atoi` and `atol` to avoid crashes due to invalid inputs.

FORBIDDEN	USE INSTEAD
gets	fgets
scanf	fgets, sscanf
fscanf	fgets, sscanf
atoi	strtol
atol	strtol

12. Documenation is mandatory. Format the documentation in Doxygen style (see Wiki and Doxygen's intro).
13. Write meaningful comments. For example, meaningful comments describe the algorithm, or why a particular solution has been chosen, if there seems to be an easier solution at a first glance. Avoid comments that just repeat the code itself
(e.g., i = i + 1; /* i is incremented by one */).
14. The documentation of a module must include: name of the module, name and student id of the author (`@author` tag), purpose of the module (`@brief`, `@details` tags) and creation date of the module (`@date` tag).

    Also the Makefile has to include a header, with author and program name at least.
15. Each function shall be documented either before the declaration or the implementation. It should include purpose (`@brief`, `@details` tags), description of parameters and return value (`@param`, `@return` tags) and description of global variables the function uses (`@details` tag).

    You should also document `static` functions (see `EXTRACT_STATIC` in the file `Doxyfile`). Document visible/exported functions in the header file and local (`static`) functions in the C file. Document variables, constants and types (especially `struct`s) too.
16. Documentation, names of variables and constants shall be in English.
17. Internal functions shall be marked with the `static` qualifier and are not allowed to be exported (e.g., in a header file). Only functions that are used by other modules shall be declared in the header file.
18. All exercises shall be solved with functions of the C standard library. If a required function is not available in the standard library, you can use other (external) functions too. Avoid reinventing the wheel (e.g., re-implementation of `strcmp`).
19. Name of constants shall be written in upper case, names of variables in lower case (maybe with fist letter capital).
20. Use meaningful variable and constant names (e.g., also semaphores and shared memories).
21. Avoid using global variables as far as possible.
22. All boundaries shall be defined as constants (macros). Avoid arbitrary boundaries. If boundaries are necessary, treat its crossing.
23. Avoid side effects with `&&` and `||`, e.g., write if (b != 0) c = a/b; instead of if (b != 0 && c = a/b).

24. Each `switch` block must contain a `default` case. If the case is not reachable, write `assert(0)` to this case (defensive programming).
25. Logical values shall be treated with logical operators, numerical values with arithmetic operators (e.g., test 2 strings for equality by strcmp (...) == 0 instead of !strcmp (...)).
26. Indent your source code consistently (there are tools for that purpose, e.g., `indent`).
27. Avoid tricky arithmetic statements. Programs are written once, but read more times. Your program is not better if it is shorter!
28. For all I/O operations (read/write from/to `stdin`, `stdout`, files, sockets, pipes, etc.) use *either* standard I/O functions (`fdopen(3)`, `fopen(3)`, `fgets(3)`, etc.) *or* POSIX functions (`open(2)`, `read(2)`, `write(2)`, etc.). Remember, standard I/O functions are buffered. Mixing standard I/O functions and POSIX functions to access a common file descriptor can lead to undefined behaviour and is therefore forbidden.
29. If asked in the assignment, you must implement signal handling (`SIGINT`, `SIGTERM`). You must only use *async-signal-safe* functions in your signal handlers.
30. Close files, free dynamically allocated memory, and remove resources after usage.
31. Don't waste resources due to inconvenient programming. Header files shall not include implementation parts (exception: macros).

Exercise 2 Guidelines

Violation of following guidelines leads to a deduction of points in exercise 2.

1. Correct use of fork/exec/pipes as tought in the lectures. For example, do not exploit inherited memory areas.
2. Ensure termination of child processes without `kill(2)` or `killpg(2)`. Collect the exit codes of child processes (`wait(2)`, `waitpid(2)`, `wait3(2)`).